

PROJET CYBERDECEPTION

DECEPTR v1 MVP

Guide complet - Architecture - Installation manuelle - Configuration - Tests - Exploitation

Champ	Valeur
Institution	Chambre des Représentants du Maroc - DSI / Cybersecurite
Projet	Plateforme de cyberdeception DECEPTR v1 MVP
Sujet	Honeypot Cowrie T-Pot, pipeline ELK, alerting et dashboard SOC
Stack	Docker, T-Pot Cowrie, Filebeat, Logstash, Redis, Python, Elasticsearch, MySQL, MinIO, Kibana, FastAPI, ElastAlert
Date	Juin 2026
Dossier final	D:\assir\ismagi\PFA\DECEPTR-FINAL

Ce document couvre

Les erreurs connues, l'architecture complete, l'installation manuelle, la configuration manuelle de chaque service, les commandes de demarrage, les tests bout-en-bout, les dashboards, la maintenance et la checklist de validation.

Sommaire

- 1. Objectif, outils a installer et ordre de construction
- 2. Erreurs et problemes connus
- 3. Architecture globale du projet
- 4. Prerequis et preparation de la machine
- 5. Construire DECEPTR de zero
- 6. Installation manuelle depuis le dossier final
- 7. Configuration manuelle des composants
- 8. Demarrage manuel de toute l'architecture
- 9. Configuration manuelle de Kibana et des dashboards
- 10. Tests utilisateur et validation bout-en-bout
- 11. Scripts d'automatisation du projet
- 12. Exploitation SOC et rapport final
- 13. Maintenance, sauvegarde et nettoyage Docker
- 14. Checklist complete
- 15. Conclusion

1. Objectif, outils a installer et ordre de construction

Cette premiere section explique ce qu'il faut installer, pourquoi chaque outil existe dans DECEPTR, et dans quel ordre construire le projet. L'objectif est de comprendre le role de chaque brique avant de lancer les commandes.

Objectif global de DECEPTR

Créer un piège crédible pour attaquants externes et utilisateurs internes malveillants: faux services, faux portail intranet, fichiers leurres, honeytokens, collecte des actions, enrichissement, scoring, alertes et dashboards SOC.

1.1 Ce qu'il faut installer sur la machine

Outil a installer	Role dans le projet	Pourquoi il est necessaire
Windows 10/11 + WSL2	Base d'execution Linux pour Docker Desktop	Permet de faire tourner les conteneurs Linux comme Elasticsearch, Logstash, Cowrie, Redis et MySQL.
Docker Desktop	Moteur de conteneurisation	Lance tous les services DECEPTR de maniere reproductible sans installer chaque serveur manuellement dans Windows.
Docker Compose	Orchestrateur local	Demarre l'architecture complete avec un seul fichier: reseaux, volumes, ports, dependances et variables.
PowerShell	Automatisation Windows	Execute `start.ps1`, `stop.ps1`, les tests E2E et les commandes de verification.
Git	Gestion de version et transport du projet	Permet de cloner, versionner et livrer le dossier projet proprement.
Python 3	Scripts de documentation et outils hors conteneur	Sert a generer DOCX/PDF, tests locaux et scripts utilitaires. Le pipeline runtime utilise Python dans Docker.
Node.js	Generation HTML des schemas	Utilise par `build-pdfs.js` pour generer les pages HTML des architectures.
GeoLite2-City.mmdb	Base locale de geolocalisation IP	Permet d'enrichir les evenements avec pays, ville et contexte reseau sans dependance externe obligatoire.
Navigateur web	Acces aux interfaces	Permet d'ouvrir Dashboard, Kibana, API Docs, MinIO et web honeypot.

1.2 Role de chaque composant DECEPTR

Etape	Composant	Travail effectue	Objectif securite
1	T-Pot Cowrie	Expose SSH/Telnet leurres, capture login, commandes et sessions	Attirer l'attaquant vers un systeme controle au lieu du vrai SI.
2	Web honeypot	Faux portail intranet avec faux documents et login capture	Pieger les tentatives d'accès applicatif interne et voler le temps de l'attaquant.
3	Dionaea	Expose FTP, HTTP, SMB et MySQL decoy	Detecter scans, malware, enumeration de services et attaques automatisées.
4	Filebeat	Lit les logs Cowrie et les envoie a Logstash	Transporter les traces sans perdre les evenements.
5	TLS 1.3	Chiffre Filebeat vers Logstash sur 5044	Eviter l'interception ou la falsification des logs en transit.
6	Logstash	Parse, structure et route les evenements	Transformer les logs bruts en donnees exploitables.
7	Redis	File d'attente `deceptr:events`	Absorber les pics de logs et decoupler ingestion/analyse.
8	Pipeline Python	Collecte, normalise, enrichit, correle, score et detecte	Transformer une trace technique en alerte SOC qualifiee.
9	Threat Intel	GeoLite2, VirusTotal, AbuseIPDB, Feodo, MITRE ATT&CK	Ajouter contexte, reputation, geolocalisation, C2 et techniques adverses.
10	Elasticsearch	Stocke logs bruts et evenements enrichis	Recherche rapide, investigation et Kibana.
11	MySQL	Stocke alertes, IoC, attackers, campaigns, honeytokens, users	Conserver les donnees metier du SOC et du dashboard.
12	MinIO	Stockage objet pour rapports, samples, backups et attachments	Garder les preuves et objets lies aux incidents.
13	FastAPI	Expose API REST securisee par JWT	Permettre au dashboard de lire alertes, stats, IoC et rapports.
14	Dashboard DECEPTR	Interface SOC simple	Voir rapidement l'etat, les alertes, IoC, campagnes et rapports.
15	Kibana	Exploration ELK avancee	Investiguer les logs bruts et creer des vues analytiques.
16	ElastAlert 2	Regles et notifications	Alerter sur les evenements critiques sans attendre une consultation manuelle.

1.3 Ordre logique de construction

- Preparer Windows, WSL2 et Docker Desktop.
- Creer le dossier final `D:\assir\ismagi\PFA\DECEPTR-FINAL`.
- Creer les reseaux Docker, volumes, `.env` et secrets.
- Installer ou lancer le runtime T-Pot/Cowrie limite au besoin du projet.
- Configurer Filebeat et les certificats TLS vers Logstash.
- Demarrer Logstash, Redis, Elasticsearch, MySQL et MinIO.
- Construire le pipeline Python et l'API FastAPI.
- Ajouter le web honeypot, Dionaea et les honeytokens.
- Configurer Kibana, Dashboard DECEPTR et ElastAlert.
- Ajouter GeoLite2 et les cles API Threat Intel si disponibles.
- Lancer le smoke test E2E puis corriger les erreurs.
- Documenter le resultat et sauvegarder les preuves.

1.4 Ce que DECEPTR doit prouver pendant la demonstration

Preuve attendue	Comment la montrer
Un attaquant peut interagir avec les leurres	Connexion SSH/Telnet Cowrie, login web honeypot ou acces Dionaea.
Les logs arrivent securises	`filebeat test output` affiche TLS 1.3 OK.
Le pipeline comprend l'attaque	Evenement enrichi dans `deceptr-events-*` avec MITRE, score et severite.
Le SOC voit l'incident	Dashboard DECEPTR affiche alertes, stats, IoC et campagnes.
Le canary detecte une fuite	`/api/canary/trigger/{token}` genere une alerte CRITICAL T1039.
Les preuves sont conservees	Elasticsearch, MySQL et MinIO contiennent les donnees.

2. Erreurs et problemes connus - Realite du terrain

Avant d'installer DECEPTR, il faut connaitre les pannes qui arrivent le plus souvent dans un lab Docker/ELK/T-Pot. Ces points correspondent aux problemes rencontres pendant la mise en route du projet.

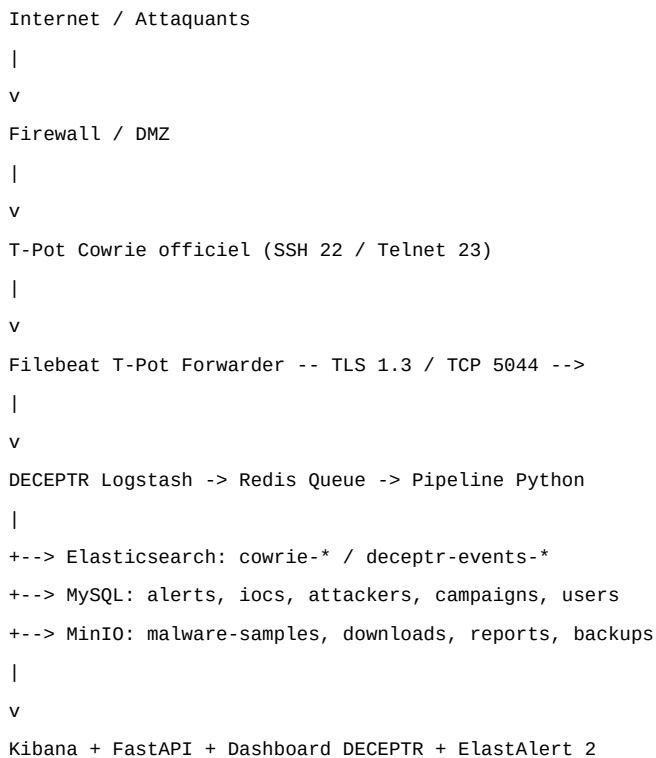
Erreur	Gravite	Symptome	Cause	Solution
Pull Docker timeout	Critique	docker compose echoue sur auth.docker.io TLS handshake timeout	Connexion lente ou Docker Hub temporairement indisponible	Relancer le pull, garder Docker Desktop ouvert, eviter VPN instable
Kibana vide	Majeur	Kibana s'ouvre mais ne montre aucun dashboard	Aucun data view cree ou aucun evenement indexe	Creer data views cowrie-* et deceptr-events-* puis generer un evenement test
Port 22 occupe	Critique	Cowrie ne demarre pas	OpenSSH Windows ou autre service utilise deja le port 22	Arreter le service SSH Windows ou changer le mapping en lab
Login dashboard ne reagit pas	Majeur	Mot de passe saisi mais rien ne se passe	API indisponible, CORS/URL API incorrecte ou MySQL non pret	Verifier http://127.0.0.1:8000/health et les logs deceptr-api
Filebeat refuse la config	Majeur	Exiting: error loading config file	Permissions strictes Filebeat	Lancer avec --strict.perms=false
TLS Filebeat vers Logstash KO	Critique	x509 ou handshake failed	CA absente ou mauvais certificat	Copier elk/certs/ca.crt dans tpot-runtime/deceptr/certs
Elasticsearch unhealthy	Critique	Kibana/API/Pipeline attendent Elasticsearch	Memoire insuffisante ou volume corrompu	Donner 4-6 GB RAM a Docker Desktop; supprimer seulement les volumes DECEPTR si necessaire
T-Pot complet trop lourd	Moyen	Beaucoup de conteneurs non prevus apparaissent	Stack T-Pot complet lance au lieu du runtime strict	Utiliser tpot-runtime avec seulement cowrie et deceptr-tpot-forwarder
Pas d'alertes	Moyen	Dashboard affiche 0 alertes	Aucun evenement Cowrie traite	Lancer le test E2E ou faire une tentative SSH/Telnet

Regle d'or

Ne pas lancer deux versions du meme DECEPTR en meme temps. Le projet final propre est D:\assir\lsmagi\PFA\DECEPTR-FINAL. Les autres projets Docker ne doivent pas etre supprimes.

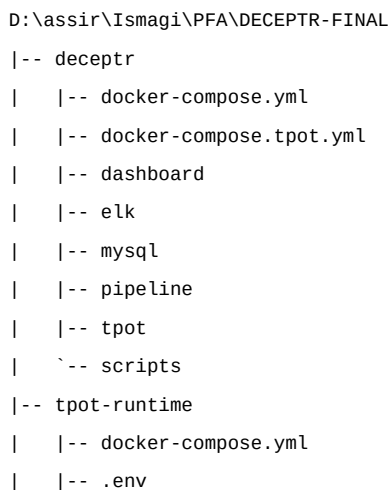
3. Architecture globale du projet

DECEPTR est une plateforme de cyberdeception. Elle expose un honeypot SSH/Telnet en DMZ, collecte les traces, enrichit les evenements, calcule un risque et presente les alertes au SOC.



Couche	Composants	Role
Detection	cowrie	Honeypot SSH/Telnet officiel T-Pot
Collecte	deceptr-tpot-forwarder	Lecture de cowrie.json et envoi TLS vers Logstash
Ingestion	deceptr-logstash	Parsing, index brut, push vers Redis
Queue	deceptr-redis	Tampon entre Logstash et le pipeline
Traitement	deceptr-pipeline	Collector, normalizer, enricher, correlator, MITRE mapper, risk scorer, alerter
Stockage	Elasticsearch, MySQL, MinIO	Logs, evenements, donnees metier, objets
Visualisation	Kibana, FastAPI, Dashboard	Exploration, API REST, interface SOC
Alerting	ElastAlert 2, alerter Python	Regles, notifications, alertes critiques

3.1 Dossier final



```
| |-- data\cowrie\log  
| `-- deceptr\filebeat + certs  
|-- docs  
|-- start.ps1  
`-- stop.ps1
```

Etat valide actuel

Le stack final ne contient pas SpiderFoot, ni ancien T-Pot web, ni services hors schema. Les conteneurs attendus sont: cowrie, deceptr-tpot-forwarder, deceptr-logstash, deceptr-redis, deceptr-pipeline, deceptr-elasticsearch, deceptr-mysql, deceptr-minio, deceptr-kibana, deceptr-api, deceptr-dashboard, deceptr-elastalert.

4. Prerequis et preparation de la machine

Element	Minimum lab	Recommande
OS	Windows 10/11 avec WSL2	Windows 11 + Docker Desktop stable
RAM Docker	4 GB	6 a 8 GB pour ELK confortable
CPU	2 coeurs	4 coeurs
Disque	20 GB libres	50 GB libres
Ports libres	22, 23, 5044, 5601, 8000, 8088, 9000, 9001, 9200	Memes ports reserves au projet
Reseau	Internet pour pull images	Connexion stable sans proxy bloquant Docker Hub

4.1 Verifications Windows

```
docker version
docker compose version
wsl --status
netstat -ano | findstr ":22"
netstat -ano | findstr ":23"
netstat -ano | findstr ":5601"
```

Si le port 22 est occupe par OpenSSH Server Windows, arreter le service avant de lancer Cowrie:

```
Get-Service sshd
Stop-Service sshd
Set-Service sshd -StartupType Manual
```

Important securite

Les mots de passe inclus sont des valeurs de lab. Pour un deploiement reel, changer ELASTIC_PASSWORD, MYSQL_PASSWORD, REDIS_PASSWORD, JWT_SECRET, MINIO_ROOT_PASSWORD et les cle API.

5. Construire DECEPTR de zero

Cette section explique comment reconstruire le projet depuis une machine vide, sans utiliser le dossier final deja prepare. Elle sert si on veut refaire le projet pour un rapport, une soutenance ou une nouvelle installation propre.

Principe

On cree d'abord l'arborescence, puis les fichiers Docker Compose, les certificats TLS, la configuration Logstash/Filebeat, la base MySQL, le pipeline Python, l'API, le dashboard, le runtime Cowrie T-Pot et enfin les scripts start/stop.

5.1 Creer l'arborescence

```
mkdir D:\assir\Ismagi\PFA\DECEPTR-FROM-ZERO
cd D:\assir\Ismagi\PFA\DECEPTR-FROM-ZERO
mkdir deceptr
mkdir deceptr\elk\certs
mkdir deceptr\elk\logstash\pipeline
mkdir deceptr\elk\logstash\config
mkdir deceptr\elk\elastalert\rules
mkdir deceptr\config\redis
mkdir deceptr\mysql
mkdir deceptr\pipeline\api\routes
mkdir deceptr\pipeline\api\models
mkdir deceptr\dashboard
mkdir deceptr\tpot\filebeat
mkdir deceptr\scripts
mkdir tpot-runtime
mkdir tpot-runtime\data\cowrie\log
mkdir tpot-runtime\deceptr\filebeat
mkdir tpot-runtime\deceptr\certs
mkdir docs
```

5.2 Creer le fichier .env

Dans deceptr\.env, mettre les secrets de lab. En production, changer toutes ces valeurs.

```
ELASTIC_PASSWORD=deceptr2025
KIBANA_SYSTEM_PASS=deceptr-kibana
REDIS_PASSWORD=deceptr-redis-2026
REDIS_HOST=redis
REDIS_PORT=6379
REDIS_QUEUE=deceptr:events
MYSQL_ROOT_PASSWORD=root2025
MYSQL_PASSWORD=mysql2025
MINIO_ROOT_USER=deceptr_admin
MINIO_ROOT_PASSWORD=MotDePasseMinIO2026!
MINIO_ENDPOINT=minio:9000
MINIO_ACCESS_KEY=deceptr_admin
MINIO_SECRET_KEY=MotDePasseMinIO2026!
MINIO_SECURE=false
JWT_SECRET=deceptr-jwt-change-me-32chars-min
VIRUSTOTAL_KEY=
ABUSEIPDB_KEY=
COWRIE_HOSTNAME=srv-prod-01
SMTP_HOST=
SMTP_PORT=587
```

```
SMTP_USER=
SMTP_PASS=
ALERT_TO=security@example.com
```

5.3 Generer les certificats TLS

```
cd D:\assir\Ismagi\PFA\DECEPTR-FROM-ZERO\deceptr\elk\certs
openssl genrsa -out ca.key 4096
openssl req -x509 -new -nodes -key ca.key -sha256 -days 3650 -out ca.crt -subj "/CN=DECEPTR-CA"
openssl genrsa -out logstash.key 2048
openssl req -new -key logstash.key -out logstash.csr -subj "/CN=logstash"
openssl x509 -req -in logstash.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out logstash.crt -
days 825 -sha256
copy ca.crt D:\assir\Ismagi\PFA\DECEPTR-FROM-ZERO\tpot-runtime\deceptr\certs\ca.crt
```

5.4 Creer docker-compose.yml DECEPTR

Le compose principal doit contenir les services suivants. On peut recopier ce fichier depuis le projet final ou le reconstruire manuellement avec cette structure.

Service	Image/build	Role	Ports
logstash	logstash:8.13.0	Ingestion Beats TLS, parsing, Redis, Elasticsearch	5044
redis	redis:7-alpine	Queue d'evenements	6379
elasticsearch	elasticsearch:8.13.0	Index cowrie-* et deceptr-events-*	9200
kibana	docker.elastic.co/kibana/kibana:8.13.0	Exploration et dashboards	5601
mysql	mysql:8.0	Donnees metier	interne 3306
minio	minio/minio:latest	Stockage objets	9000/9001
pipeline	build ./pipeline	Traitement Python	interne
api	build ./pipeline	FastAPI	8000
dashboard	nginx:1.27-alpine	Interface SOC statique	8088
elastalert	jertel/elastalert2:latest	Regles alerting	interne

Creer aussi docker-compose.tpot.yml pour desactiver les services locaux cowrie/filebeat du compose principal:

```
services:
  cowrie:
  profiles: ["local-cowrie"]
  filebeat:
  profiles: ["local-filebeat"]
```

5.5 Creer le runtime Cowrie T-Pot

Dans tpot-runtime\docker-compose.yml, garder seulement Cowrie officiel T-Pot et Filebeat forwarder.

```
networks:
  cowrie_local:
  deceptr_internal:
external: true
services:
  cowrie:
  image: ghcr.io/telekom-security/cowrie:24.04.1
  container_name: cowrie
  restart: unless-stopped
  networks: [cowrie_local]
  ports:
  - "22:22"
```

```

- "23:23"
read_only: true
volumes:
- ./data/cowrie/downloads:/home/cowrie/cowrie/dl
- ./data/cowrie/keys:/home/cowrie/cowrie/etc
- ./data/cowrie/log:/home/cowrie/cowrie/log
- ./data/cowrie/log/tty:/home/cowrie/cowrie/log/tty
deceptr-tpot-forwarder:
image: elastic/filebeat:8.13.0
container_name: deceptr-tpot-forwarder
restart: unless-stopped
user: root
command: ["--strict.perms=false"]
environment:
- DECEPTR_LOGSTASH_HOST=logstash
- DECEPTR_LOGSTASH_PORT=5044
networks:
- cowrie_local
- deceptr_internal
volumes:
- ./deceptr/filebeat/tpot-cowrie-to-deceptr.yml:/usr/share/filebeat/filebeat.yml:ro
- ./deceptr/certs/ca.crt:/usr/share/filebeat/certs/ca.crt:ro
- ./data/cowrie/log:/tpot/cowrie/log:ro

```

5.6 Creer Filebeat T-Pot

Fichier: `deceptr\tpot\filebeat\tpot-cowrie-to-deceptr.yml` puis copie vers `tpot-runtime\deceptr\filebeat`.

```

filebeat.inputs:
- type: log
enabled: true
paths:
- /tpot/cowrie/log/cowrie.json
json.keys_under_root: true
json.overwrite_keys: true
json.add_error_key: true
tags: ["cowrie", "tpot"]
fields:
deceptr_source: tpot-cowrie
output.logstash:
hosts: ["${DECEPTR_LOGSTASH_HOST}:${DECEPTR_LOGSTASH_PORT}"]
ssl.enabled: true
ssl.certificate_authorities: ["/usr/share/filebeat/certs/ca.crt"]
ssl.verification_mode: full
logging.level: warning

```

5.7 Creer Logstash

Fichier: `deceptr\elk\logstash\pipeline\cowrie.conf`. Il doit recevoir Beats TLS, parser JSON, indexer cowrie-* et pousser vers Redis.

```

input {
beats {
port => 5044
ssl_enabled => true

```

```

ssl_certificate => "/usr/share/logstash/config/certs/logstash.crt"
ssl_key => "/usr/share/logstash/config/certs/logstash.key"
}
}
filter {
  if [eventid] {
    mutate { add_field => { "deceptr_pipeline" => "cowrie" } }
  }
}
output {
  elasticsearch {
    hosts => ["http://elasticsearch:9200"]
    user => "elastic"
    password => "${ELASTIC_PASSWORD}"
    index => "cowrie-%{+YYYY.MM}"
  }
  redis {
    host => "redis"
    port => 6379
    password => "${REDIS_PASSWORD}"
    data_type => "list"
    key => "${REDIS_QUEUE}"
  }
}

```

5.8 Creer MySQL schema.sql

Le schema minimal doit contenir users, alerts, iocs, attackers et campaigns.

```

CREATE DATABASE IF NOT EXISTS deceptr;
USE deceptr;
CREATE TABLE IF NOT EXISTS users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(80) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role ENUM('admin', 'soc_analyst', 'threat_hunter', 'auditeur') DEFAULT 'soc_analyst',
  active TINYINT(1) DEFAULT 1,
  last_login DATETIME NULL
);
CREATE TABLE IF NOT EXISTS alerts (
  id INT AUTO_INCREMENT PRIMARY KEY,
  event_id VARCHAR(128),
  source_ip VARCHAR(64),
  severity VARCHAR(32),
  risk_score INT,
  mitre_technique VARCHAR(64),
  description TEXT,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE IF NOT EXISTS iocs (
  id INT AUTO_INCREMENT PRIMARY KEY,
  indicator VARCHAR(255),
  type VARCHAR(64),

```

```

reputation VARCHAR(64),
first_seen DATETIME DEFAULT CURRENT_TIMESTAMP,
last_seen DATETIME DEFAULT CURRENT_TIMESTAMP
);

```

5.9 Creer le pipeline Python

Le pipeline peut etre minimal au debut, puis enrichi. Les modules obligatoires sont:

Module	Responsabilite minimale
collector.py	Lire Redis et retourner les evenements
normalizer.py	Transformer Cowrie en champs standards: source.ip, username, password, event.type
enricher.py	Ajouter geoip/reputation si les cles existent
correlator.py	Regrouper par IP et campagne
risk_scorer.py	Calculer score et severite
detector.py	Creer alerte si login_attempt ou brute force
storage.py	Ecrire dans Elasticsearch, MySQL et MinIO
main.py	Boucle principale
api/main.py	FastAPI health, auth, alerts, stats

Dockerfile minimal du pipeline/API:

```

FROM python:3.11-slim
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends gcc curl && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN useradd -r -u 1001 -g root deceptr && chown -R deceptr:root /app
USER deceptr

```

requirements.txt minimal:

```

fastapi
uvicorn[standard]
elasticsearch
redis
mysql-connector-python
python-jose[cryptography]
passlib[bcrypt]
requests
minio
geoip2

```

5.10 Creer dashboard et API

- Creer api/main.py avec /health, /api/auth/login, /api/alerts, /api/alerts/stats.
- Creer dashboard/index.html qui appelle http://127.0.0.1:8000.
- Verifier que le formulaire login envoie username/password vers /api/auth/login.
- Afficher KPIs, timeline, alertes recentes et IoC.

5.11 Creer start.ps1 et stop.ps1

```

# start.ps1
$Root = Split-Path -Parent $MyInvocation.MyCommand.Path

```

```

$DeceptrRoot = Join-Path $Root "deceptr"
$TpotRoot = Join-Path $Root "tpot-runtime"
Copy-Item -Force "$DeceptrRoot\tpot\filebeat\tpot-cowrie-to-deceptr.yml" "$TpotRoot\deceptr\
filebeat\tpot-cowrie-to-deceptr.yml"
Copy-Item -Force "$DeceptrRoot\elk\certs\ca.crt" "$TpotRoot\deceptr\certs\ca.crt"
Push-Location $DeceptrRoot
docker compose -f docker-compose.yml -f docker-compose.tpot.yml up -d --build
Pop-Location
Push-Location $TpotRoot
docker compose up -d cowrie deceptr-tpot-forwarder
Pop-Location
# stop.ps1
Push-Location .\tpot-runtime
docker compose stop deceptr-tpot-forwarder cowrie
Pop-Location
Push-Location .\deceptr
docker compose -f docker-compose.yml -f docker-compose.tpot.yml stop
Pop-Location

```

5.12 Validation d'un build zero

```

cd D:\assir\Ismagi\PFA\DECEPTR-FROM-ZERO
powershell -ExecutionPolicy Bypass -File .\start.ps1
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
docker exec deceptr-tpot-forwarder filebeat test output -e --strict.perms=false
curl.exe -s -o NUL -w "%{http_code}" http://127.0.0.1:8000/health
curl.exe -s -o NUL -w "%{http_code}" http://127.0.0.1:8088/index.html

```

Critere de reussite

Un build zero est valide quand Cowrie tourne sur 22/23, Filebeat annonce TLSv1.3 OK, l'API retourne 200, le dashboard retourne 200, Kibana repond, et un evenement test arrive dans cowrie-* puis deceptr-events-*.

6. Installation manuelle depuis le dossier final

6.1 Recuperer le dossier final

Le dossier final a été consolidé dans un seul répertoire. Pour une installation manuelle sur une autre machine, copier ce dossier complet sans séparer `deceptr` et `tpot-runtime`.

```
D:\assir\Ismagi\PFA\DECEPTR-FINAL
```

6.2 Installer Docker Desktop

- Installer Docker Desktop pour Windows.
- Activer WSL2 backend dans Settings > General.
- Dans Settings > Resources, donner au minimum 4 GB RAM.
- Démarrer Docker Desktop et attendre que l'icône indique Running.
- Tester docker version et docker compose version.

6.3 Charger les images manuellement

Si on veut vérifier image par image avant de lancer compose:

```
docker pull ghcr.io/telekom-security/cowrie:24.04.1
docker pull elastic/filebeat:8.13.0
docker pull logstash:8.13.0
docker pull elasticsearch:8.13.0
docker pull docker.elastic.co/kibana/kibana:8.13.0
docker pull redis:7-alpine
docker pull mysql:8.0
docker pull minio/minio:latest
docker pull nginx:1.27-alpine
docker pull jertel/elastalert2:latest
```

6.4 Construire les images Python

```
cd D:\assir\Ismagi\PFA\DECEPTR-FINAL\deceptr
docker compose -f docker-compose.yml -f docker-compose.tpot.yml build api pipeline
```


7. Configuration manuelle des composants

7.1 Fichier .env principal

Chemin: D:\assir\ismagi\PFA\DECEPTR-FINAL\deceptr\.env

Variable	Role	Valeur lab actuelle / remarque
ELASTIC_PASSWORD	Mot de passe elastic	deceptr2025
KIBANA_SYSTEM_PASS	Compte technique Kibana	deceptr-kibana
REDIS_PASSWORD	Protection Redis	deceptr-redis-2026
MYSQL_ROOT_PASSWORD	Root MySQL	root2025
MYSQL_PASSWORD	Utilisateur deceptr MySQL	mysql2025
MINIO_ROOT_USER	Compte console MinIO	deceptr_admin
MINIO_ROOT_PASSWORD	Mot de passe MinIO	MotDePasseMinIO2026!
JWT_SECRET	Signature tokens API	Changer en production
VIRUSTOTAL_KEY / ABUSEIPDB_KEY	Threat intelligence	Optionnel; garder vide si pas de cle
ALERT_TO	Email destinataire alertes	security@example.com par default

Ne pas publier le .env

Le fichier .env peut contenir des secrets. Il doit rester local au lab ou etre remplace par un gestionnaire de secrets en production.

7.2 Certificats TLS Filebeat -> Logstash

Le projet contient deja les certificats dans deceptr\elk\certs. Pour regenerer manuellement:

```
cd D:\assir\ismagi\PFA\DECEPTR-FINAL\deceptr\elk\certs
openssl genrsa -out ca.key 4096
openssl req -x509 -new -nodes -key ca.key -sha256 -days 3650 -out ca.crt -subj "/CN=DECEPTR-CA"
openssl genrsa -out logstash.key 2048
openssl req -new -key logstash.key -out logstash.csr -subj "/CN=logstash"
openssl x509 -req -in logstash.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out logstash.crt -
days 825 -sha256
```

Copier ensuite la CA vers le runtime T-Pot:

```
copy .\ca.crt D:\assir\ismagi\PFA\DECEPTR-FINAL\tpot-runtime\deceptr\certs\ca.crt
```

7.3 T-Pot Cowrie et Filebeat

Chemin: tpot-runtime\docker-compose.yml. Ce runtime est volontairement strict: seulement Cowrie officiel T-Pot et Filebeat forwarder.

Service	Configuration critique
cowrie	Image ghcr.io/telekom-security/cowrie:24.04.1, ports 22:22 et 23:23, logs dans tpot-runtime\data\cowrie\log
deceptr-tpot-forwarder	Image elastic/filebeat:8.13.0, lit /tpot/cowrie/log/cowrie.json, envoie vers logstash:5044 en TLS

7.4 Logstash

Logstash ecoute sur TCP/5044 en Beats TLS. Il indexe les logs bruts dans cowrie-YYYY.MM et pousse une copie dans Redis.

```
docker logs deceptr-logstash --tail 80
docker exec deceptr-tpot-forwarder filebeat test output -e --strict.perms=false
```

7.5 Pipeline Python

Fichier	Role
collector.py	Consomme Redis puis fallback Elasticsearch cowrie-*
normalizer.py	Transforme Cowrie brut en schema DECEPTR unifie
enricher.py	Ajoute GeoIP, VirusTotal, AbuseIPDB, Feodo
correlator.py	Regroupe evenements et IoC
risk_scorer.py	Calcule le score 0-100
detector.py	Declenche les regles de detection
alerter.py	Genere alertes et notifications
storage.py	Ecrit Elasticsearch, MySQL et MinIO
api/main.py	Expose FastAPI et les endpoints dashboard

7.6 Comptes par default

Interface	URL	Utilisateur	Mot de passe lab
Dashboard DECEPTR	http://127.0.0.1:8088/index.html?v=3	admin	deceptr2025
Kibana	http://127.0.0.1:5601	elastic	deceptr2025
API Docs	http://127.0.0.1:8000/docs	JWT via /api/auth/login	admin / deceptr2025
MinIO	http://127.0.0.1:9001	deceptr_admin	MotDePasseMinIO2026!

8. Demarrage manuel de toute l'architecture

8.1 Methode recommandee

```
cd D:\assir\Ismagi\PFA\DECEPTR-FINAL
powershell -ExecutionPolicy Bypass -File .\start.ps1
```

8.2 Methode totalement manuelle

Etape 1 - preparer les fichiers Filebeat et certificats:

```
mkdir D:\assir\Ismagi\PFA\DECEPTR-FINAL\tpot-runtime\deceptr\filebeat
mkdir D:\assir\Ismagi\PFA\DECEPTR-FINAL\tpot-runtime\deceptr\certs
copy D:\assir\Ismagi\PFA\DECEPTR-FINAL\deceptr\tpot\filebeat\tpot-cowrie-to-deceptr.yml D:\assir\Ismagi\PFA\DECEPTR-FINAL\tpot-runtime\deceptr\filebeat\
copy D:\assir\Ismagi\PFA\DECEPTR-FINAL\deceptr\elk\certs\ca.crt D:\assir\Ismagi\PFA\DECEPTR-FINAL\tpot-runtime\deceptr\certs\
mkdir D:\assir\Ismagi\PFA\DECEPTR-FINAL\tpot-runtime\data\cowrie\log
```

Etape 2 - demarrer DECEPTR sans Cowrie local:

```
cd D:\assir\Ismagi\PFA\DECEPTR-FINAL\deceptr
docker compose -f docker-compose.yml -f docker-compose.tpot.yml up -d --build
```

Etape 3 - demarrer Cowrie T-Pot et le forwarder:

```
cd D:\assir\Ismagi\PFA\DECEPTR-FINAL\tpot-runtime
docker compose up -d cowrie deceptr-tpot-forwarder
```

8.3 Verifier l'etat

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
curl.exe -s -o NUL -w "%{http_code}" http://127.0.0.1:8088/index.html?v=3
curl.exe -s -o NUL -w "%{http_code}" http://127.0.0.1:8000/health
curl.exe -s -o NUL -w "%{http_code}" http://127.0.0.1:5601
docker exec deceptr-tpot-forwarder filebeat test output -e --strict.perms=false
```

Test	Resultat attendu
Dashboard	HTTP 200
API health	HTTP 200
Kibana	HTTP 302 ou page login
Filebeat test output	TLS handshake OK, TLSv1.3
Cowrie	Ports 22 et 23 publies

9. Configuration manuelle de Kibana et des dashboards

- Ouvrir <http://127.0.0.1:5601>.
- Se connecter avec elastic / deceptr2025.
- Aller dans Stack Management > Data Views.
- Créer le data view cowrie-* avec le champ temps @timestamp.
- Créer le data view deceptr-events-* avec le champ temps @timestamp.
- Aller dans Discover et vérifier que les événements apparaissent.

Dashboard Kibana conseillé	Visualisation	Source
Activite Cowrie	Histogramme par heure	cowrie-*
Top IP attaquantes	Table terms source.ip	deceptr-events-*
Identifiants testés	Table username/password	deceptr-events-*
MITRE ATT&CK	Bar chart mitre.technique	deceptr-events-*
Severite	Donut severity	deceptr-events-*
Carte GeoIP	Map source.geo.location	deceptr-events-*

Pourquoi Kibana peut sembler vide

Kibana n'invente pas les données. Il faut au moins un événement Cowrie traité dans cowrie-* ou deceptr-events-* avant que Discover et Lens affichent quelque chose.

10. Tests utilisateur et validation bout-en-bout

10.1 Test interactif SSH/Telnet

```
ssh root@127.0.0.1 -p 22
# Essayer quelques mots de passe: admin, 123456, password
telnet 127.0.0.1 23
# Essayer login: admin / password: admin
```

Ces actions doivent créer des lignes dans `tpot-runtime\data\cowrie\log\cowrie.json`.

10.2 Test E2E automatisé

```
powershell -ExecutionPolicy Bypass -File D:\assir\Ismagi\PFA\DECEPTR-FINAL\deceptr\scripts\e2e-smoke.ps1 -TpotRoot D:\assir\Ismagi\PFA\DECEPTR-FINAL\tpot-runtime -WaitSeconds 75
```

Champ attendu	Exemple valide
status	OK
tpot_cowrie	running
tpot_forwarder_tls	TLSv1.3
raw_index	cowrie-2026.06
enriched_index	deceptr-events-2026.06
enriched_type	login_attempt
mitre	T1110

10.3 Verification API

```
$body = @{username='admin'; password='deceptr2025'} | ConvertTo-Json
$token = (Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/api/auth/login -
    ContentType 'application/json' -Body $body).access_token
Invoke-RestMethod -Uri http://127.0.0.1:8000/api/alerts -Headers @{Authorization="Bearer
    $token"}
Invoke-RestMethod -Uri http://127.0.0.1:8000/api/alerts/stats -Headers @{Authorization="Bearer
    $token"}
```

11. Scripts d'automatisation du projet

Script	Emplacement	Role
start.ps1	DECEPTR-FINAL	Prepare fichiers, démarre DECEPTR, Cowrie T-Pot et forwarder
stop.ps1	DECEPTR-FINAL	Arrete les services du projet sans supprimer les volumes
e2e-smoke.ps1	deceptr/scripts	Injecte un evenement et verifie toute la chaine
start-architecture.ps1	deceptr/scripts	Redirection vers le start final
stop-architecture.ps1	deceptr/scripts	Redirection vers le stop final

11.1 Script start.ps1

Le script fait trois choses: verifie Docker, copie la configuration Filebeat/certificats vers tpot-runtime, puis lance les deux compose.

```
cd D:\assir\Ismagi\PFA\DECEPTR-FINAL
powershell -ExecutionPolicy Bypass -File .\start.ps1
```

11.2 Script stop.ps1

```
cd D:\assir\Ismagi\PFA\DECEPTR-FINAL
powershell -ExecutionPolicy Bypass -File .\stop.ps1
```

12. Exploitation SOC et rapport final

Le SOC exploite DECEPTR via trois niveaux: dashboard rapide, Kibana pour investigation, API pour extraction et integration.

Besoin SOC	Outil	Action
Voir le niveau de menace	Dashboard DECEPTR	Ouvrir /index.html?v=3 et verifier KPIs
Analyser une IP	API / Kibana	Filtrer source.ip puis voir credentials, pays, MITRE
Voir brute force	Kibana	Filtrer event.type=login_attempt et MITRE T1110
Exporter un rapport	API DGSSI	Consulter /api/dgssi/rapport ou /api/dgssi/rapport/json
Traiter une alerte	Dashboard/API	Acknowledge et documenter la decision

12.1 Structure du rapport final

Section	Contenu
Resume executif	Nombre d'alertes, severite dominante, tendance 24h, decision a prendre
Architecture	Schema reseau, fonctionnel et technique
Sources de donnees	Cowrie, Filebeat, Logstash, Redis, pipeline
Alertes	Liste par severite, IP, MITRE, score
IoC	IP, reputation, geolocalisation, historique
Timeline	Evenements par heure et par campagne
Recommandations	Court terme, moyen terme, durcissement
Annexes	Commandes, hashes, versions, captures dashboard

13. Maintenance, sauvegarde et nettoyage Docker

13.1 Sauvegarder les donnees

```
docker run --rm -v decepтр_elastic_data:/data -v D:\backups:/backup alpine tar czf /backup/elastic_data.tgz /data
```

```
docker run --rm -v decepтр_mysql_data:/data -v D:\backups:/backup alpine tar czf /backup/mysql_data.tgz /data
```

```
docker run --rm -v decepтр_minio_data:/data -v D:\backups:/backup alpine tar czf /backup/minio_data.tgz /data
```

13.2 Nettoyer uniquement DECEPTR

Ne jamais lancer docker system prune sans comprendre l'impact. Pour ce projet, supprimer seulement les conteneurs nommes DECEPTR si necessaire:

```
docker rm -f cowrie decepтр-tpot-forwarder decepтр-logstash decepтр-redis decepтр-pipeline  
decepтр-elasticsearch decepтр-mysql decepтр-minio decepтр-kibana decepтр-api decepтр-dashboard  
decepтр-elastalert
```

Attention

Ne pas supprimer les projets non lies comme Pentagi, SIEM, Mongo ou autres environnements Docker personnels.

13.3 Logs utiles

```
docker logs cowrie --tail 80
```

```
docker logs decepтр-tpot-forwarder --tail 80
```

```
docker logs decepтр-logstash --tail 120
```

```
docker logs decepтр-pipeline --tail 120
```

```
docker logs decepтр-api --tail 120
```

```
docker logs decepтр-kibana --tail 120
```


14. Checklist complete

Phase	Point de controle	OK
Preparation	Docker Desktop démarre et docker version répond	[]
Preparation	Ports 22, 23, 5044, 5601, 8000, 8088, 9000, 9001, 9200 libres	[]
Preparation	Dossier final présent dans D:\assir\smagi\PFA\DECEPTR-FINAL	[]
Configuration	.env rempli avec mots de passe lab ou production	[]
Configuration	ca.crt copie vers tpot-runtime\deceptr\certs	[]
Demarrage	start.ps1 termine sans erreur	[]
Conteneurs	12 conteneurs attendus visibles dans docker ps	[]
TLS	filebeat test output indique TLSv1.3 OK	[]
Dashboard	http://127.0.0.1:8088/index.html?v=3 retourne 200	[]
API	http://127.0.0.1:8000/health retourne 200	[]
Kibana	http://127.0.0.1:5601 retourne 302 ou login	[]
Kibana	Data views cowrie-* et deceptr-events-* créés	[]
Honeypot	Tentative SSH ou Telnet génère cowrie.json	[]
Pipeline	e2e-smoke.ps1 retourne status OK	[]
Rapport	Captures ou exports disponibles pour le rapport PFA	[]

15. Conclusion

DECEPTR v1 MVP repond a l'objectif du projet: deployer une plateforme de cyberdeception reproductible, dockerisee, basee sur Cowrie T-Pot et un pipeline complet de collecte, enrichissement, detection, stockage, visualisation et alerting.

Objectif	Resultat
Honeypot reel	Cowrie officiel T-Pot expose sur SSH 22 et Telnet 23
Collecte securisee	Filebeat vers Logstash en TLS 1.3
Pipeline complet	Redis + Python normalisation/enrichissement/correlation/MITRE/risk/alertes
Stockage	Elasticsearch, MySQL et MinIO
Visualisation	Kibana, Dashboard DECEPTR et API FastAPI
Alerting	ElastAlert 2 et alerter Python
Reproductibilite	Un seul dossier final avec start.ps1 et stop.ps1

Projet pret

La partie logicielle correspond au schema MVP. La partie restante pour une production reelle est l'infrastructure physique: firewall pfSense/FortiGate, segmentation DMZ/LAN/SOC, VPN/MFA, certificats publics et politique de sauvegarde.